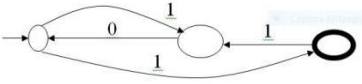




Matrícula	Nome:
-----------	-------

Parte I. Questão 1 – Responda (nesse mesmo arquivo) falso, verdade ou NS (não sei). Duas erradas anulam uma certa. Branco conta como errada. Cada resposta correta recebe 0,4 pontos. **Em caso da resposta ser falso, justifique porque ela é falsa.** Boa prova [6,0]

1. A análise semântica mantém a tabela de símbolos, detecta a maioria dos erros e, se não houver erros, gera código intermediário.	V
2. Nas linguagens compiladas atuais, a análise semântica também é a responsável por eliminar comentários e delimitadores redundantes, expandir macros e fazer a montagem condicional.	F
3. A análise sintática utilizar <i>scanners</i> modelados como FSA para identificar os <i>tokens</i> do programa.	F
4. As semânticas possíveis para $A \leftarrow 2 + 3$ são atribuir valor 5 ou operação “2+3” para A. Com checagem de tipo dinâmica, o tipo de A decide a semântica. Com checagem estática, o tipo de A depende do valor a ele atribuído.	F
5. BNF é uma gramática livre de contexto, simples e poderosa que consegue expressar regras sintáticas com dependência contextual.	F
6. A string 101101 é reconhecida pelo autômato 	V
7. A string arara pode ser reconhecida por um autômato <i>push down</i> determinístico.	V
8. Na fase de síntese do código, constituem exemplos de otimização algorítmica: efetuar cálculos de valores comuns, eliminar atribuições constantes em loops, efetuar cálculo de subscritos, e eliminar variáveis temporárias.	NS
9. O conjunto de linguagens aceitas por FSA é equivalente às linguagens geradas por gramáticas ou expressões regulares.	V
10. O <i>parser</i> grupa sequência de caracteres em constituintes elementares do fonte (símbolos) na análise léxica.	F
11. Regras sintáticas definidas por uma sintaxe formal (gramática) são suficientes para determinar a interpretação correta de uma sentença.	F
12. Sintaxe de uma LP é o conjunto de regras que determinam quando uma sentença é bem formada.	V
13. Um PDA é um FSA com uma pilha associada que pode ser utilizado para reconhecer <i>strings</i> em uma linguagem com sentenças geradas por regras de uma gramática regular.	V
14. Uma gramática regular possui regras da forma: <não-terminal> ::= terminal <não-terminal> terminal.	V
15. Uma linguagem é qualquer conjunto de <i>strings</i> (de tamanho finito) de caracteres escolhidos de um alfabeto fixo de símbolos finitos.	V

Justificativas para as respostas respondidas como **falso**. Indique o número da pergunta e a seguir apresente a sua justificativa.

- Os comentários são removidos durante a fase de análise léxica, e portanto não chegam a análise semântica para serem processados. Os delimitadores redundantes são ignorados durante processo de obtenção dos tokens.
- O scanner é executado na análise léxica, e não na sintática.
- Na checagem estática, o tipo é declarado juntamente com a definição da variável, e não devido a valor atribuído.
- Não é possível expressar regras sintáticas com dependência contextual através da BNF.
- O parser não faz parte da análise léxica.
- Não são suficientes, pois a linguagem muitas das vezes irá depender de contexto e outras regras como a associatividade da soma, ou prioridade de multiplicação e divisão em relação a outras operações.



Matrícula	Nome:
-----------	-------

Parte II. Responda a **duas** das questões a a seguir. Caso deseje, utilize uma folha de papel para fazer desenhos, digitalize e inclua a imagem no arquivo Word. **Não utilize exemplos vistos em sala de aula.**

Questão 2 – Porque é difícil para um tradutor determinar a igualdade de dois objetos de dados do mesmo tipo (0,5)? Exemplifique (0,5). Qual foi a abordagem para afirmar que os tipos dos dois objetos do seu exemplo são equivalentes (1,0)? [2,0]

É difícil para um tradutor saber se dois objetos do mesmo tipo são iguais pois isto depende principalmente do contexto para o qual aquele objeto atende. Caso seja utilizado tipos primários é algo relativamente simples, dado que os bits armazenados serão iguais. Para um objeto, no entanto, igualdade pode assumir diversos significados, e portanto cabe ao programador descrever o seu significado.

Como exemplo, temos a estrutura pessoa abaixo:

```
class Pessoa {  
public:  
    string nome;  
    int cpf;  
    bool equals(Pessoa* outra) {  
        return outra->cpf == this->cpf;  
    }  
}
```

E as definições ‘Pessoa* pessoa1 = new Pessoa(“Mateus Silva”, 1548932100)’ e ‘Pessoa* pessoa2 = new Pessoa(“Mateus Silva”, 1548932100)’.

Em determinados contextos, é possível que o programador deseje comparar os CPFs armazenados, e em outros queira determinar se os objetos estão na mesma posição de memória (Java com o Object.Equals).

No meu exemplo, estou definindo que a operação de igualdade verificará CPFs, tendo em vista que cada pessoa só pode possuir um CPF no sistema brasileiro, e que cada CPF só pode pertencer a um único usuário. Em C++, cada comparação entre classes (ou si mesma) requisita que o programador escreva código.

Questão 3 – Escreva um código C com programa principal e função recursiva chamada por ele (0,5). Descreva o suporte necessário e como funciona a execução de uma função recursiva. (1,0). Apresente o conteúdo do segmento de código e do registro de ativação ao final da segunda chamada da função recursiva (1,0). [2,5]

Questão 4 – Apresente um exemplo (e informe o significado) gerado por cada um dos seguintes formalismos: BNF, EBNF, FSA, gramática regular e expressão regular. (0,5 cada) [2,5]

BNF significa Backus-Naur Form, e é uma linguagem formal que pode ser utilizada para descrever outras linguagens (é portanto uma metalinguagem). Ela estabelece diversos conceitos básicos (“::=”, “bicudos > e <”, “[]”) e exige uma definição rígida de todos os elementos. Exemplo:

```
<nascimento> ::= <mes>/<ano>  
<mes> ::= <digito><digito>  
<ano> ::= <digito><digito><digito><digito>
```

Com tais regras, “11/2020” será considerado nascimento.

EBNF é uma extensão da BNF padrão, adicionando novos elementos básicos como o fecho de Kleene (que indica repetição de zero ou mais elementos na sintaxe), elementos opcionais (elementos “cobertos” por colchetes). Exemplo:

```
<nascimento> ::= {<digito>}2 / {<digito>}4
```

A expressão significa “nascimento é dois dígitos (mês), seguidos de uma barra, seguidos de 4 dígitos (ano)”. E pode identificar 10/1987 corretamente como nascimento.

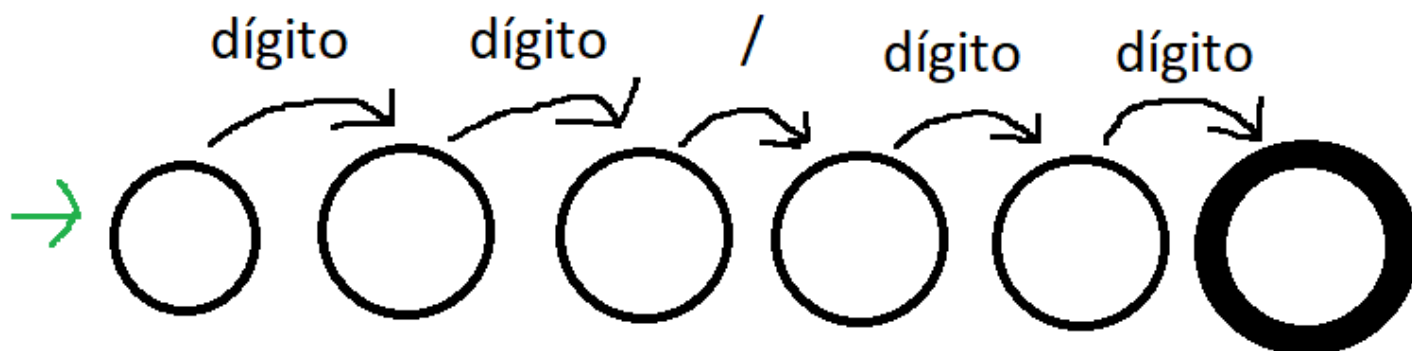


Matrícula	Nome:
-----------	-------

Um FSA é um autômato de estados finitos, usado principalmente na obtenção de tokens. Por exemplo, podemos usar um FSA para identificar dia de nascimento:

Queremos que o FSA identifique $\langle \text{dígito} \rangle \langle \text{dígito} \rangle / \langle \text{dígito} \rangle \langle \text{dígito} \rangle$ (ou seja, DD/MM)

Caso seja lido próximo registro da string em quaisquer das etapas um registro não presente, o autômato fica em um estado preso “estado trap” não podendo chegar ao estado final desejado.



Gramáticas Regulares: são todas as gramáticas que podem ser expressas através de $\langle \text{não-terminal} \rangle ::= \text{terminal} \langle \text{não-terminal} \rangle \mid \text{terminal}$, ou seja utilizando-se desta regra em específico podem ser geradas todas as possibilidades da gramática.

Expressões Regulares: aplicação de expressões lógicas em definição. Pode ser representado $\langle a \rangle \vee \langle b \rangle$, por exemplo, que significa “a” ou “b”. São extremamente poderosas para a conversão para uma visualização gráfica de um nó para outro. Exemplo:

01(1 ∨ 2)0*

